MMDU MULLANA OFFICIAL SYLLABUS • BCSE-007 • UNIT 4

Unit 4: Trees, Binary Search Trees & Graph Algorithms

Subject: Data Structures & Algorithms (BCSE-007) • **Exam Weightage:** 25 - 30 Marks • **Study Time:** 45 mins

Tree terminology, binary tree theorems, Binary Search Tree (BST) operations (Search, Insert, Deletion 3 cases), Inorder/Preorder/Postorder traversals, Expression trees, Graph representations (Adjacency Matrix vs List), and BFS/DFS traversal algorithms with cycle detection.

1. Binary Tree Properties & Theorems

1. Core Mathematical Theorems on Binary Trees

1. **Maximum nodes at level i** (root at level 0):

$N_{max}(i) = 2^i$

2. **Maximum nodes in a binary tree of height h:**

$N_{max} = \Sigma_{i=0}^{h} 2^i = 2^{(h+1)} - 1$

3. **Minimum height of a binary tree with n nodes:**

$h_{min} = \lceil \log_2(n + 1) \rceil - 1$

4. **Strictly Binary Tree Relation:** In any non-empty strictly binary tree, the number of leaf nodes L is always one more than the number of internal nodes with two children I:

$L = I + 1$

2. Binary Tree Taxonomy

- **Full (Proper) Binary Tree:** Every node has either 0 or 2 children.
- **Complete Binary Tree:** All levels are completely filled except possibly the last, which is filled sequentially from left to right.
- **Perfect Binary Tree:** All internal nodes have 2 children and all leaf nodes reside at the exact same depth.

2. Binary Search Tree (BST) Operations & Traversals

1. The BST Invariant Property

For every node X in a Binary Search Tree:
key(LeftSubtree(X)) < key(X) < key(RightSubtree(X))

2. BST Traversal Orders

- **Inorder (Left, Root, Right):** Recursively traverses left, visits root, traverses right. **Crucial Property:** Inorder traversal of a BST always yields keys in strictly ascending sorted order!
- **Preorder (Root, Left, Right):** Used for creating tree duplicates / prefix serialization.
- **Postorder (Left, Right, Root):** Used for deleting trees / evaluating expression trees bottom-up.

3. BST Node Deletion (The 3 Canonical Cases)

1. **Case 1 (Leaf Node):** Target node has zero children. Simply set parent's reference to NULL and free(node).
2. **Case 2 (Node with One Child):** Bypass the node by linking its parent directly to its sole child.
3. **Case 3 (Node with Two Children):**

- Find the node's **Inorder Successor** (smallest key in right subtree, i.e., leftmost node in right child) OR **Inorder Predecessor** (largest key in left subtree).
- Copy the successor's data value into the target node.
- Recursively delete the inorder successor (which is guaranteed to have at most 1 child!).

3. Graph Representations & Search Algorithms (BFS & DFS)

1. Graph Representations: Matrix vs List

Feature	Adjacency Matrix	Adjacency List
Data Structure	2D Array int adj[V][V]	Array of Linked Lists Node* adj[V]
Space Complexity	$\Theta(V^2)$ (Fixed memory)	$\Theta(V + E)$ (Optimal for sparse graphs)
Edge Lookup (u, v)	O(1) direct array index	O(degree(u)) linked traversal
Find All Neighbors	O(V) scans whole row	O(degree(u)) traverses neighbor list
Best Suited For	**Dense Graphs** ($E \approx V^2$)	**Sparse Graphs** ($E \ll V^2$)

2. Breadth-First Search (BFS)

- **Mechanism:** Level-order exploration using a **FIFO Queue** and a visited boolean array.
- **Applications:** Shortest path in unweighted graphs, minimum spanning trees, connected components.
- **Time Complexity:** O(V + E) with adjacency list; Space: O(V) queue.

3. Depth-First Search (DFS)

- **Mechanism:** Deep path exploration using a **LIFO Stack** (or runtime recursion) and backtracking when dead ends are reached.
- **Applications:** Cycle detection in directed/undirected graphs, Topological Sort (DAG), finding strongly connected components (Kosaraju/Tarjan).
- **Time Complexity:** O(V + E) with adjacency list; Space: O(V) call stack.